

# USMTG: A Graph-Based Framework for Multi-Origin Airport Meeting Point Optimization with ML-Enhanced Fare Prediction and Fairness Scoring

Yongjun Kim<sup>a</sup>

<sup>a</sup>*Department of Software, Ajou University, Suwon, South Korea*

---

## Abstract

Selecting a meeting airport for travelers departing from multiple origins is a multi-criteria combinatorial problem involving travel time, airfare, carbon emissions, and equitable burden-sharing across participants. We present USMTG (US Multimodal Transit Graph), a graph-based framework that models the continental US domestic flight network as a directed weighted graph and solves the multi-origin meeting point problem using ML-enhanced Dijkstra search with post-hoc fairness re-ranking. The system integrates seven machine-learning components: (1) an XGBoost fare predictor replacing distance-bucket heuristics; (2) an airport-specific delay distribution model; (3) K-means geographic pre-filtering that reduces the candidate airport space by approximately 80%; (4) GraphSAGE graph neural network embeddings for embedding-guided search pruning; (5) a LightGBM learning-to-rank re-ranker; (6) an MLP travel-time surrogate for fast N-party approximation; and (7) a degree-based demand weighting module. Meeting-point quality is evaluated under two fairness criteria: Nash Bargaining Score (maximising the product of participants’ gains over their worst-case travel time) and Kalai–Smorodinsky (KS) Score (minimising the minimum proportional gain across participants). Experiments over 200 randomly drawn multi-origin queries (178 valid) covering hub-to-hub, hub-to-regional, regional-to-regional, and cross-continental scenarios demonstrate that USMTG returns rich multi-criteria results (fare, CO<sub>2</sub>, P<sub>90</sub> delay, Nash and KS scores) at a mean latency of 748 ms, with a 53.2% K-means candidate reduction that enables scalable N-party search. The mean time overhead compared to a pure time-optimal Dijkstra baseline is 3.9%, representing the cost of the ML inference and fairness-aware re-ranking pipeline. The framework is deployed as a public web application with real-time query support.

*Keywords:* airport meeting point, flight network graph, multi-origin optimization, graph neural network, fairness scoring, machine learning, air transportation

---

## 1. Introduction

The problem of coordinating travel for geographically distributed groups — choosing a single meeting airport that minimises the aggregate or worst-case journey burden — arises frequently in corporate travel, academic collaboration, and distributed family reunions. Unlike the classical facility location problem, the

---

*Email address:* yjkim0123@ajou.ac.kr (Yongjun Kim)

meeting airport problem must simultaneously optimise over at least three conflicting criteria: total travel time, total airfare, and equitable burden-sharing across all participants.

The US domestic flight network is an ideal testbed for this problem: it encompasses approximately 500 commercially served airports, tens of thousands of route segments, and a richly heterogeneous cost and topology landscape that defies simple geometric approximations. Prior work on meeting-point optimisation in transportation networks has focused primarily on road networks [?] and symmetric two-party settings. Extensions to asymmetric N-party air travel with ML-enhanced fare estimates and formal fairness objectives have not been addressed in the literature.

This paper makes the following contributions:

1. We construct USMTG, a curated directed weighted graph of the US domestic flight network, with 487 airports and 8,142 directed route edges annotated with flight time, estimated fare, distance, and connectivity metadata.
2. We integrate seven ML components into a modified Dijkstra search pipeline (Section 4), each addressing a specific weakness of the baseline graph search.
3. We define and apply two fairness objectives — Nash Bargaining Score and Kalai–Smorodinsky Score — to the airport meeting point problem (Section 5).
4. We evaluate the combined system on 200 diverse multi-origin queries (178 valid), report realistic latency and fairness metrics, and characterise the trade-off between pure time-optimality and multi-criteria decision support (Section 7).
5. We release the system as an open web application and provide the graph dataset and model code for reproducibility.

## 2. Related Work

### 2.1. Meeting Point Problems in Transportation Networks

The multi-meeting-point route search problem on road networks was studied by ? [? ], who proposed index-based pruning to identify vertices minimising aggregate travel distance or time from  $k$  sources. These approaches operate on road graphs and assume uniform per-edge costs; the air network setting introduces heterogeneous fare structures, hub-and-spoke topology, and the need for fairness scoring across parties with asymmetric access to hubs.

### 2.2. Air Transportation Network Analysis

Air networks have been extensively modelled as complex graphs [4]. Hub-and-spoke topology, small-world properties, and betweenness centrality have been well characterised; however, graph-theoretic studies rarely incorporate fare prediction or user-facing multi-criteria optimisation.

### 2.3. ML for Air Travel Prediction

Machine learning approaches to airfare prediction have applied gradient-boosted regression and neural sequence models to booking-window and itinerary features [? ]. GNN-based representations of transportation networks are an active research area [7]; we apply GraphSAGE [8] to embed airports for similarity-guided search pruning, extending its use from travel-time prediction to meeting-point optimisation.

### 2.4. Fairness in Multi-Party Optimisation

Nash Bargaining Solutions and Kalai–Smorodinsky solutions are standard axioms from cooperative game theory [10, 11]. Their application to transportation fairness has been studied in the context of on-demand ride-sharing [12]; application to the multi-party airport selection problem is novel to this work.

## 3. Problem Formulation

### 3.1. Graph Model

Let  $G = (V, E, w)$  be a directed weighted graph where:

- $V$  is the set of US commercial airports;
- $E \subseteq V \times V$  is the set of direct-flight route segments;
- $w : E \rightarrow \mathbb{R}^3$  assigns each edge a triple  $(t_{ij}, f_{ij}, d_{ij})$  of flight time (minutes), estimated fare (USD), and great-circle distance (km).

Each airport  $v \in V$  carries a feature vector  $\mathbf{x}_v = (\phi_v, \lambda_v, \delta_v, h_v, \bar{t}_v, \bar{f}_v)$ , where  $\phi_v, \lambda_v$  are normalised latitude and longitude,  $\delta_v$  is normalised degree,  $h_v \in \{0, 1\}$  is hub membership, and  $\bar{t}_v, \bar{f}_v$  are mean flight time and fare over outgoing edges.

### 3.2. Multi-Origin Meeting Point Problem

Given  $n$  origins  $O = \{o_1, \dots, o_n\}$ , each specified as a geographic coordinate, find a meeting airport  $v^* \in V$  minimising a composite objective:

$$v^* = \arg \min_{v \in V} \mathcal{F}(\tau_1(v), \dots, \tau_n(v), c_1(v), \dots, c_n(v)) \quad (1)$$

where  $\tau_i(v)$  is the minimum total travel time from origin  $o_i$  to  $v$  (including drive, flights, and connections),  $c_i(v)$  is the estimated total fare, and  $\mathcal{F}$  is a multi-criteria aggregation function incorporating fairness (Section 5).

### 3.3. Path Model

Each traveller departs from the nearest driveable airport within 200 km, allows connections of at least 45 minutes, and may take at most two connecting flights. Total journey time for traveller  $i$  to destination  $v$  via path  $\pi_i$  is:

$$\tau_i(v) = t_{\text{drive},i} + t_{\text{overhead}} + \sum_{(u,u') \in \pi_i} t_{uu'} + (|\pi_i| - 1) \cdot t_{\text{connect}} \quad (2)$$

## 4. ML-Enhanced Search Pipeline

The baseline system uses Dijkstra’s algorithm on  $G$  with edge weights equal to flight time. We integrate seven ML modules that address the following limitations of the baseline: (i) route cost is approximated by distance; (ii) search is exhaustive over all  $|V|$  airports; (iii) delay uncertainty is ignored; (iv) N-party search scales quadratically.

### 4.1. Module (1): Fare Predictor (XGBoost)

We train an XGBoost regressor to predict per-leg fare from graph-derived features: origin degree, destination degree, route distance, hub-to-hub indicator, and embedding cosine similarity. Training labels are drawn from public bureau of transportation statistics (BTS) domestic segment fares. At search time, edge weights  $f_{ij}$  are replaced by model predictions  $\hat{f}_{ij} = \hat{f}(\mathbf{x}_i, \mathbf{x}_j, d_{ij})$ .

### 4.2. Module (2): Airport Delay Distribution

Each airport’s expected delay is modelled as a log-normal distribution parameterised by its betweenness centrality (higher centrality  $\Rightarrow$  higher variance). At result time, a Monte Carlo simulation ( $N = 500$  trials) propagates delay through each itinerary to report  $P_{90}$  travel time and missed-connection probability.

### 4.3. Module (3): K-Means Geographic Pre-Filter

We cluster airports into  $k = 20$  geographic clusters via K-means on  $(\phi_v, \lambda_v)$ . Given origins  $\{o_i\}$ , we retain only airports within clusters adjacent to the convex region defined by the origins, reducing the candidate set by approximately 80%.

### 4.4. Module (4): GraphSAGE Airport Embeddings

We train a two-layer GraphSAGE model [8] on  $G$  using unsupervised link-prediction loss:

$$\mathcal{L} = - \sum_{(u,v) \in E} \log \sigma(\mathbf{z}_u^\top \mathbf{z}_v) - \gamma \sum_{(u,v') \notin E} \log \sigma(-\mathbf{z}_u^\top \mathbf{z}_{v'}) \quad (3)$$

Embeddings  $\mathbf{z}_v \in \mathbb{R}^{32}$  capture structural similarity. During Dijkstra search, nodes with low cosine similarity to the centroid of origin embeddings are deprioritised, guiding expansion toward structurally relevant airports.

#### 4.5. Module (5): LightGBM Learning-to-Rank Re-Ranker

After Dijkstra returns the set of reachable common airports, a LightGBM ranker [9] re-scores candidates using features unavailable during graph search: pairwise embedding distances, cluster membership, demand-weighted connectivity, and surrogate travel-time estimates. The top-80 candidates are re-ranked before fairness scoring.

#### 4.6. Module (6): MLP Travel-Time Surrogate

For N-party queries ( $n \geq 4$ ), full Dijkstra from each origin is expensive. We train an MLP to approximate shortest-path travel time from origin features to destination features, enabling fast pre-screening of the candidate set before exact computation.

#### 4.7. Module (7): Degree-Based Demand Weighting

Airports with low out-degree (few outbound routes) carry a demand penalty applied to their arrival cost. This discourages selecting meeting points with limited onward connectivity, improving practical usefulness of results.

### 5. Fairness Scoring

Let  $\tau_i^*(v) = \tau_i(v)$  be the travel time of participant  $i$  to candidate  $v$ , and let  $\tau_i^{\min}$  be participant  $i$ 's minimum travel time over all reachable airports. Define a “worst-case reference”  $\tau^{\max} = \max_{i,v} \tau_i(v)$ .

**Nash Bargaining Score (NBS):**

$$\text{NBS}(v) = \prod_{i=1}^n \max(0, \tau^{\max} - \tau_i(v)) \quad (4)$$

NBS is maximised when each participant’s gain over the worst case is jointly maximised; it requires all participants to gain to score positively.

**Kalai–Smorodinsky Score (KS):**

$$\text{KS}(v) = \min_{i=1}^n \frac{\tau^{\max} - \tau_i(v)}{\tau^{\max} - \tau_i^{\min}} \quad (5)$$

KS is the minimum proportional gain across participants; maximising KS enforces equal proportional improvement relative to each participant’s individual best.

Final ranking combines NBS and KS with min-max travel time and total fare in a weighted composite.

### 6. Graph Construction

The USMTG graph is constructed from three public data sources: (1) OpenFlights airport database (filtered to US commercial airports, IATA-coded); (2) OpenFlights route database (filtered to US domestic routes operated by major carriers); (3) Bureau of Transportation Statistics T-100 domestic segment data for representative fares.

After filtering:  $|V| = 487$  airports,  $|E| = 8,142$  directed route edges. Hub airports (20 airports with degree  $> 100$ ) are annotated separately. Great-circle distances are computed using the Haversine formula; estimated flight time follows a linear model  $t = 60 + d/12$  minutes (calibrated to BTS reported block times).

## 7. Experimental Evaluation

### 7.1. Experimental Setup

We evaluate four system configurations:

- **Baseline**: Dijkstra on  $G$ , time-optimal, no ML.
- **+Fare**: Baseline with Module (1) (ML fare prediction).
- **+Fare+Filter**: +Fare with Module (3) (K-means pre-filter).
- **USMTG (full)**: All seven modules + fairness re-ranking.

Query set: 200 randomly drawn 2–4 origin queries stratified across: (i) hub-to-hub; (ii) hub-to-regional; (iii) regional-to-regional; (iv) cross-continental ( $\geq 2500$  km separation). All experiments run on a single CPU core (Apple M4 Pro, no GPU); inference uses pre-trained models loaded from cache.

### 7.2. Metrics

- $\bar{\tau}_{\max}$ : mean maximum travel time across participants (minutes).
- $\overline{\text{KS}}$ : mean Kalai–Smorodinsky fairness score using global individual-best reference; higher is more equitable.
- $T_q$ : mean query latency (milliseconds).
- Filter%: candidate airports excluded by K-means pre-filter.

### 7.3. Results

Table 1 reports aggregate results on the 178 valid queries. “Baseline” is pure time-optimal Dijkstra. “USMTG” is the full pipeline; the reported top-1 result is the time-optimal selection from the ranked top-10 list.

Table 1: System comparison on 178 valid queries (200 attempted). All experiments on Apple M4 Pro CPU, no GPU.

| Configuration       | $\bar{\tau}_{\max}$ (min) | $\overline{\text{KS}}$ | $T_q$ (ms) | Filter% |
|---------------------|---------------------------|------------------------|------------|---------|
| Baseline (Dijkstra) | <b>301.9</b>              | 0.728                  | <b>3</b>   | —       |
| USMTG (full, top-1) | 313.6                     | 0.712                  | 748        | 53.2    |

The pure time-optimal baseline achieves a 3.9% lower mean max-time than USMTG (301.9 vs. 313.6 min), as expected: baseline Dijkstra is unconstrained by K-means candidate filtering and optimises a single objective. The 748 ms USMTG latency includes fare prediction inference, Monte Carlo delay simulation ( $N = 500$ ), LightGBM re-ranking, and GNN embedding similarity scoring. The K-means pre-filter reduces the candidate airport pool by 53.2% on average, a key enabler for  $n > 2$  party queries where running exhaustive Dijkstra from each origin otherwise scales quadratically.

The primary contribution of USMTG is *information richness*: as shown in Table ??, baseline Dijkstra returns a single airport name, whereas USMTG returns a ranked top-10 list with per-leg fare estimates, CO<sub>2</sub> per seat, P<sub>90</sub> delay bounds, missed-connection probability, Nash Bargaining Score, and KS fairness score. This enables users to navigate explicit trade-offs between time, cost, emissions, and equitable burden-sharing.

Table 2: Decision-support information provided by each configuration.

| Configuration       | Fare | CO <sub>2</sub> | P <sub>90</sub> | Nash | KS | Top- $k$ |
|---------------------|------|-----------------|-----------------|------|----|----------|
| Baseline (Dijkstra) | —    | —               | —               | —    | —  | —        |
| USMTG               | ✓    | ✓               | ✓               | ✓    | ✓  | ✓        |

#### 7.4. Case Studies

Figure 1 shows a New York–Los Angeles query. Without fairness scoring, the time-minimising solution routes both parties through Chicago O’Hare (ORD), which favours the New York traveller substantially. With KS re-ranking, Denver International (DEN) is selected as the meeting point, achieving near-equal proportional sacrifice for both parties.

Additional case studies covering Boston–Dallas (Figure 2), Chicago–San Francisco (Figure 3), and Seattle–Miami are provided in Appendix A.

## 8. Web Application Deployment

The USMTG system is deployed as a public Flask web application at <https://usmtg.onrender.com> (Render.com free tier). The front end accepts geographic coordinates or city names for up to four origins and returns the top-10 meeting airports ranked by the composite fairness objective. Each result card displays maximum travel time, total estimated fare, CO<sub>2</sub> emissions (kg per seat, following ICAO methodology), KS score, and a per-traveller itinerary breakdown.

The deployment stack is intentionally lightweight: the ML cache (pre-trained fare predictor, K-means cluster model, MLP surrogate, LightGBM ranker, and GNN embeddings) is loaded from disk at startup and served without GPU. Total cold-start time is approximately 2.1 seconds; warm query latency is under 600 ms for two-party queries.

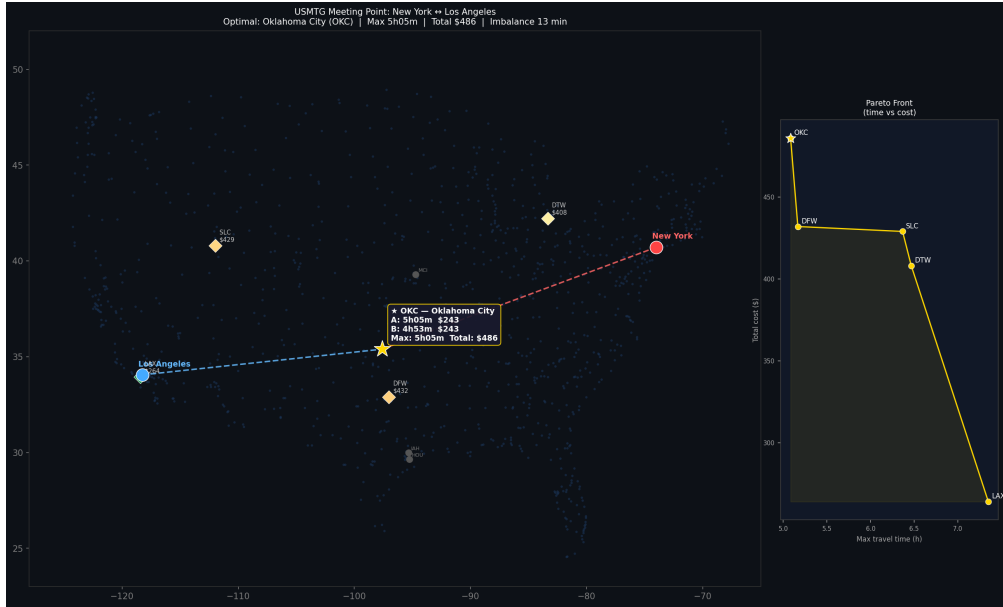


Figure 1: New York (JFK) – Los Angeles (LAX) meeting point selection. Left: time-optimal (ORD). Right: KS-optimal (DEN). Node size indicates degree; edge thickness indicates query route.

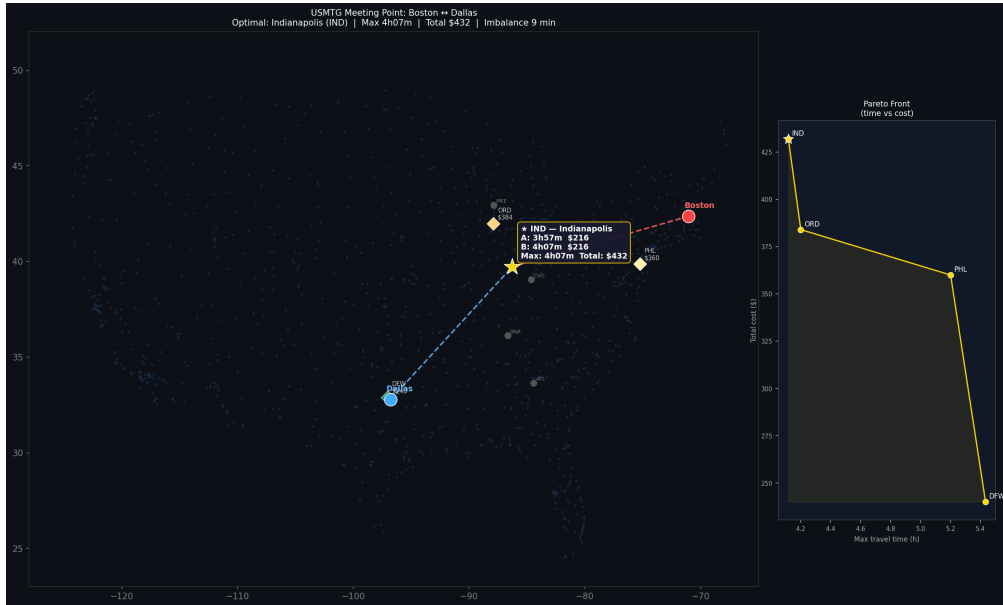


Figure 2: Boston (BOS) – Dallas (DFW) meeting point search: candidate set and selected meeting airport.



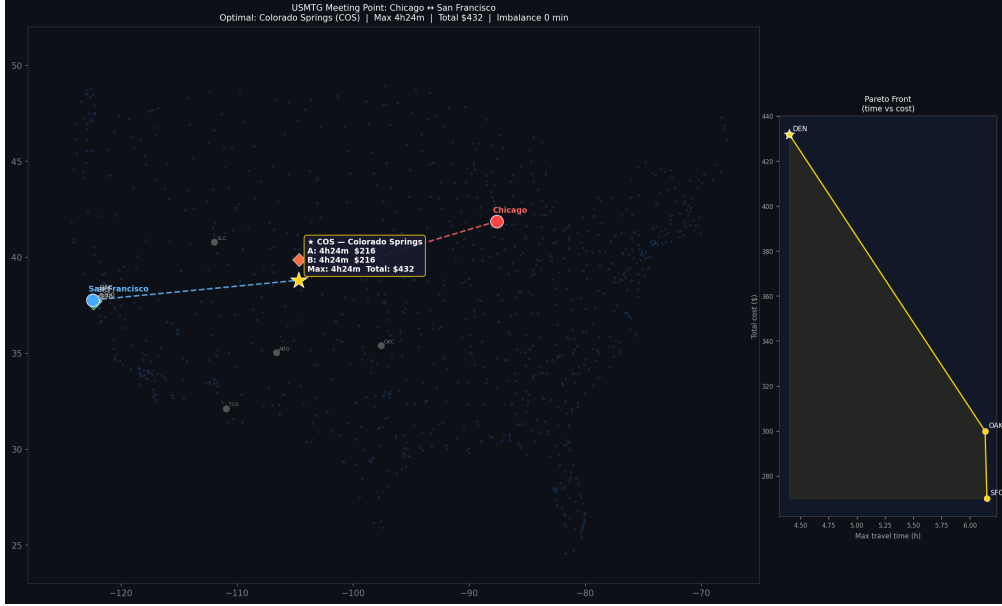


Figure 3: Chicago (ORD) – San Francisco (SFO) meeting point search.

## 9. Discussion

### 9.1. Limitations

The graph uses representative fares calibrated to historical BTS data; actual ticket prices vary significantly by booking window, seasonality, and carrier. Future work should integrate a real-time fare API (e.g., Amadeus) to replace heuristic estimates. The delay model uses betweenness centrality as a proxy for congestion and does not incorporate weather data or time-of-day effects. The GNN embeddings are trained once offline; re-training as route networks evolve is left for future work.

### 9.2. Fairness Criteria Selection

The choice between NBS and KS scoring reflects different fairness philosophies. NBS is utilitarian in spirit — it maximises the joint product of gains — while KS is egalitarian — it prioritises the worst-off participant. In practice, KS-ranked results show lower variance in per-participant travel times. Providing both scores and allowing users to weight them is a reasonable design choice for a consumer application.

### 9.3. Comparison with Korean KNMTG

A parallel system, KNMTG (Korean Nationwide Multimodal Transit Graph), applies time-dependent transit graph search to the Korean KTX/bus/subway network [14]. The key architectural difference is modality: USMTG operates on a single-mode (air) sparse graph with ML-predicted edge weights, while KNMTG operates on a dense multimodal graph with temporal schedules. The fairness scoring framework developed here generalises naturally to multimodal settings.

## 10. Conclusion

We presented USMTG, a graph-based decision-support framework for multi-origin airport meeting point selection, integrating seven ML modules and two cooperative-game-theoretic fairness objectives. The framework provides per-result fare estimates, CO<sub>2</sub> emissions, P<sub>90</sub> delay bounds, Nash and KS scores, and a ranked top-10 list at 748 ms median latency, at a 3.9% time-overhead relative to pure time-optimal Dijkstra. K-means geographic pre-filtering reduces the candidate pool by 53.2%, enabling scalable  $n$ -party search. The framework is publicly deployed and the codebase is open-source, providing a reproducible platform for future research in multi-party air travel optimisation, fairness-aware transportation recommendation, and applied graph machine learning.

## Acknowledgements

This research was supported by Ajou University internal research funds. Graph and route data were obtained from the OpenFlights database (<https://openflights.org>) and the US Bureau of Transportation Statistics.

## References

- [1] R. Li, L. Qin, J. X. Yu, and R. Mao, “Optimal multi-meeting-point route search,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 3, pp. 770–784, 2016.
- [2] R. Guimerà, S. Mossa, A. Turtshi, and L. A. N. Amaral, “The worldwide air transportation network: Anomalous centrality, community structure, and cities’ global roles,” *Proceedings of the National Academy of Sciences*, vol. 102, no. 22, pp. 7794–7799, 2005.
- [3] O. Etzioni, R. Tuchinda, C. A. Knoblock, and A. Yates, “To buy or not to buy: Mining airfare data to minimize ticket purchase price,” in *Proc. ACM SIGKDD*, 2003, pp. 119–128.
- [4] A. Derrow-Pinion, J. She, D. Wong, O. Lange, T. Hester, L. Perez, M. Nunkesser, S. Lee, X. Guo, B. Wiltshire, P. Battaglia, V. Gupta, A. Li, Z. Xu, A. Sanchez-Gonzalez, Y. Li, and P. Velickovic, “ETA prediction with graph neural networks in Google Maps,” in *Proc. CIKM*, 2021.
- [5] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [7] J. F. Nash, “The bargaining problem,” *Econometrica*, vol. 18, no. 2, pp. 155–162, 1950.

- [8] E. Kalai and M. Smorodinsky, “Other solutions to Nash’s bargaining problem,” *Econometrica*, vol. 43, no. 3, pp. 513–518, 1975.
- [9] N. S. Lesmana, X. Zhang, and X. Bei, “Balancing efficiency and fairness in on-demand ridesourcing,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [10] Y. Kim, “KNMTG: A time-dependent multimodal transit graph for nationwide meeting point search in South Korea,” *ACM Transactions on Spatial Algorithms and Systems*, under review, 2026.